

전자전기컴퓨터설계실험Ⅱ

실험 전 보고서

LAB6

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 10월 25일
대상	UART 통신과 FPGA 응용
검증 환경	VS Code / Vivado

1. 실험 목적과 관련 이론

실험 목적

PC와 STM32, FPGA 사이에서 UART로 데이터를 전달하고, 입력한 문자가 수신 장치에 도착하는지 확인한다. 과제 5에서는 문자와 바이트 값을 비교하고, 과제 6에서는 MCU를 경유한 FPGA 수신을, 과제 7에서는 UART 명령에 따른 FPGA LED 제어를 검증할 계획이다.[1]

FPGA 개별 실습은 서보모터 제어, SRAM 읽기·쓰기, DAC 순서로 진행한다. 이 보고서는 설계와 예상 결과를 정리한 실험 전 보고서이며, 실제 파형과 시연 결과는 실험 후 기록한다.

UART 프레임과 전송 시간

UART는 송수신 장치가 공통 클록선 없이 정해진 속도로 통신하는 비동기 직렬 방식이다. 이번 설정은 9600 bps, 데이터 8비트, 패리티 없음, 정지 1비트인 8N1이다. 유휴 상태는 HIGH이며 시작 비트 1개, 데이터 8개, 정지 비트 1개로 한 바이트에 10비트 시간이 필요하다.[1]

비트 시간은 1/9600초로 약 104.17 μs이며, 한 프레임은 약 1.042 ms이다. 추가 지연이 없는 연속 전송의 이론상 상한은 초당 960바이트이다. 이번 코드는 수신 후 안내문을 출력하므로 한 문자씩 보내고 응답을 확인한다.

실험 장치와 통신 설정

항목	설정 및 역할
MCU	NUCLEO-G474RE
FPGA	실습용 FPGA 보드
USART2	PA2 TX / PA3 RX ST-LINK VCP와 PC 연결
USART3	PC10 TX / PC11 RX FPGA 통신
공통 설정	9600 / 8N1 / 흐름 제어 없음
PC 전송	줄바꿈 None, 한 문자씩

UART 초기화는 main.c, 핀 연결은 HAL MSP 코드에서 확인한다. HAL 드라이버는 주변장치 설정과 송수신 함수를 제공한다.[1]

2. 과제 5 PC와 MCU 입출력

설계 목표와 회로 구성

PC를 ST-LINK USB에 연결하고 MCU의 VCP COM 포트를 연다. 이 과제에서는 FPGA와 점퍼선이 필요하지 않다. PC에서 보낸 문자 1개를 MCU가 받고, 같은 문자와 16진수 바이트 값을 PC에 출력하도록 설계한다.[1]

핵심 코드와 입출력 연결

ece2_474_stdio.c의 표준 입력은 UART2 수신으로, 표준 출력은 UART2 송신으로 연결된다. getchar()는 EOF도 표현해야 하므로 결과를 int로 받은 뒤 유효한 값만 uint8_t로 변환한다.[1]

```
int ch = getchar();
if (ch == EOF) {
    clearerr(stdin);
    return;
}
uint8_t data = (uint8_t)ch;
pc_last_byte = data;
++pc_rx_count;
printf("RX: '%c' (0x%02X)",
       (int)data,
       (unsigned int)data);
```

위 코드는 출력 가능한 문자 처리 부분의 발췌이다. 실제 코드는 제어 문자와 비ASCII 입력을 문자 대신 설명과 바이트 값으로 표시한다. 초기화에서 표준 입출력 버퍼링을 끄며 UART2 인터럽트 수신은 함께 사용하지 않는다.

예상 결과

입력	바이트	예상 출력
1	0x31	'1' (0x31)
A	0x41	'A' (0x41)
a	0x61	'a' (0x61)

문자 1은 숫자 값 1인 0x01과 다르다. 대문자 A와 소문자 a도 서로 다른 바이트로 표시되어야 한다. 입력이 없으면 getchar()에서 대기하며, 이를 프로그램 오류로 해석하지 않는다.

검증 계획

과제 5 실행 구성을 선택하고 F5로 시작한다. main에서 멈추면 실행을 재개한 뒤 1, A, a를 차례로 전송한다. 터미널 결과와 pc_last_byte, pc_rx_count를 비교하고 화면을 저장한다. 연속 입력으로 문자가 누락되면 먼저 한 문자씩 전송하여 기본 경로부터 확인한다.

3 과제 6 MCU와 FPGA 통신

설계 목표와 회로 구성

PC의 MCU COM 포트로 문자를 보내면 STM32가 UART3로 전달하고, FPGA가 자체 USB-UART를 통해 PC에 같은 문자를 출력하도록 구성한다. MCU로 응답을 되돌리는 경로가 아니라 FPGA의 별도 COM 포트에서 수신을 확인하는 방식이다.[1]

구간	연결
PC → MCU	ST-LINK USB / UART2
MCU → FPGA	PC10 TX → 확장 GPIO RX
공통 기준	MCU GND ↔ FPGA GND
FPGA → PC	F6 TX → 내부 USB-UART
PC11 RX	이번 단방향 전달에는 미사용

FPGA의 기존 RX C6는 내부 USB-UART에 연결된 핀이므로 점퍼선 입력과 구분한다. 실제 사용할 확장 GPIO와 패키지 핀을 교재 핀맵으로 확인한 뒤 FPGA XDC의 RX를 변경한다. MCU 핀 설정은 제공 IOC와 생성 코드를 사용한다.[1]

전달 핵심 코드

```
uint8_t data = (uint8_t)ch;
HAL_StatusTypeDef result =
    HAL_UART_Transmit(
        &huart3, &data, 1U, 100U);
fpga_last_tx_status = result;
```

수신한 바이트를 변환 없이 전달한다. 1U는 한 바이트, 100U는 최대 대기 시간 100 ms이다. 송신 후 결과가 HAL_OK이면 성공 횟수를, 나머지는 오류 횟수를 증가시켜 디버거에서 확인한다.[1][2]

반환값	의미
0	HAL_OK 정상 종료
1	HAL_ERROR 오류
2	HAL_BUSY 사용 중
3	HAL_TIMEOUT 시간 초과

예상 결과와 검증 계획

서로 다른 두 COM 포트를 9600 8N1로 열고 MCU 포트에 1, A, a를 보낸다. FPGA 포트에도 같은 문자가 나타나야 한다. MCU 송신 결과 0은 송신 함수의 완료만 뜻하므로 FPGA 출력과 반드시 비교한다.

수신이 없으면 TX/RX 연결, GND, XDC, FPGA 비트스트림, COM 선택을 순서대로 확인한다.

4. 과제 7 UART로 LED 제어

설계 목표와 회로 구성

과제 6의 연결을 유지하면서 FPGA의 수신 완료 이벤트로 LED 상태를 갱신한다. MCU는 과제 6과 동일하게 모든 문자를 전달한다. LED 명령 해석은 FPGA에서 수행하며 수신 문자도 FPGA COM 포트에 출력하여 명령과 실제 상태를 함께 확인한다.[1]

명령	바이트	동작
0	0x30	LED 끄기
1	0x31	LED 켜기
2	0x32	상태 반전
기타	해당 바이트	이전 상태 유지

FPGA 명령 처리 설계

다음은 수신 완료 신호 rx_valid가 한 클록 펄스라는 조건에서의 설계 예시다. 실제 UART 모듈과 포트 이름에 맞춰 연결하고, 초기 LED 상태는 꺼짐으로 정의한다.

```
always @(posedge clk) begin
  if (rst)
    led_state <= 1'b0;
  else if (rx_valid) begin
    case (rx_data)
      8'h30: led_state <= 1'b0;
      8'h31: led_state <= 1'b1;
      8'h32: led_state <=
                          ~led_state;
      default: ;
    endcase
  end
end
```

수신 완료 펄스 없이 매 클록마다 2를 처리하면 여러 번 반전할 수 있다. 논리 상태 led_state와 실제 핀 출력 극성은 구분하며, LED가 active-low이면 출력단에서 반전한다. 핀 번호는 사용할 보드의 LED 핀맵에 맞춰 지정한다.

반복 명령의 예상 결과

입력 순서	기대 LED
0 → 1 → 1	꺼짐 → 켜짐 → 켜짐
2 → 2	꺼짐 → 켜짐
A → 0	켜짐 유지 → 꺼짐

표는 첫 행부터 연속 전송하는 조건이다. 특히 1을 반복해도 계속 켜져 있고, 2를 보낼 때마다 한 번씩 반전되는지 관찰한다. 임의 문자와 줄바꿈에는 상태가 바뀌지 않아야 한다.

5. FPGA 서보모터 제어

설계 목표와 회로 구성

servo_position에서 목표 펄스 폭을 정하고 servo_pwm에서 PWM을 생성하며 servo_ctrl이 두 모듈을 연결한다. 50 MHz 클록을 기준으로 20 ms 프레임을 만들고 HIGH 폭을 1~2 ms에서 0.1 ms씩 바꾼다. 초기 폭은 1.5 ms이다.[2]

left_pulse와 right_pulse에는 버튼 동기화·디바운스 후 한 클록 펄스를 넣는다. 버튼을 직접 연결하면 한 번 누르는 동안 여러 클록에 걸쳐 값이 변할 수 있다. 서보 제어 출력과 스텝모터의 4상 출력은 서로 다른 핀이므로 서보 전용 핀맵을 사용한다.

위치 변경 핵심 코드

```
if (rst_p)
  pulse_us <= 1500;
else if (left_pulse &&
         !right_pulse &&
         pulse_us > 1000)
  pulse_us <= pulse_us - 100;
else if (right_pulse &&
         !left_pulse &&
         pulse_us < 2000)
  pulse_us <= pulse_us + 100;
```

동시 입력은 두 조건이 모두 거짓이므로 현재 폭을 유지한다. 하한과 상한에서 더 누르더라도 범위를 벗어나지 않는다. PWM 폭은 프레임 경계에서만 갱신하여 진행 중인 펄스가 중간에 잘리지 않도록 한다.

예상 파형

항목	시간	50 MHz 클록 수
프레임	20 ms	1,000,000
최소 폭	1.0 ms	50,000
초기 폭	1.5 ms	75,000
최대 폭	2.0 ms	100,000
변경 단위	0.1 ms	5,000

듀티비는 5%, 7.5%, 10%에 해당한다. 펄스 폭에 따라 축 위치가 변할 것으로 예상되지만 정확한 각도와 회전 방향은 모터 특성 및 장착 상태에 따라 실제로 확인한다.

서보 테스트벤치

교안 TB는 계산용 CLK_HZ를 100,000으로 줄여 프레임 2,000클록, HIGH 100·150·200클록을 검사한다. 실제 TB 클록 주기는 10 ns이므로 표시 시간축을 보드의 20 ms로 해석하지 않고 클록 수를 비교한다. 리셋 시 목표 폭 1,500 µs와 출력 LOW, 왼쪽·오른쪽 반복 입력의 한계, 동시 입력 시 유지, 재리셋을 확인한다. 보드에서는 20 ms 주기를 유지하면서 HIGH 폭이 바뀌는지 오실로스코프로 확인하고 축 위치를 기록한다.

6. FPGA SRAM 읽기와 쓰기

설계 목표와 회로 구성

같은 주소에 데이터를 쓰고 다시 읽어 값이 보존되는지 확인한다. sram_bus_control은 읽기·쓰기와 버스 방향을 결정하고 sram은 주소, 데이터, LED를 연결한다. 주소 버스는 18비트, 데이터 버스는 16비트지만 이번 예제는 주소 0~15와 데이터 하위 4비트만 사용한다.

버스 제어 핵심 코드

```
assign drive_data =
    ctrl_w && !ctrl_r;
assign read_data =
    ctrl_r && !ctrl_w;
assign ram_cs =
    !(drive_data || read_data);
assign ram_oe = !read_data;
assign ram_we = !drive_data;
assign ram_data = drive_data
    ? {12'b0, data_in} : 16'bz;
```

읽을 때 FPGA 출력은 고임피던스 Z가 되어 SRAM이 버스를 구동할 수 있도록 한다. 동시에 두 장치가 다른 값을 구동하면 버스 충돌이 발생하므로 방향 제어가 중요하다. 읽기·쓰기 동시 요청은 둘 다 수행하지 않도록 설계되어 있다.

W/R	CS/OE/WE	동작
0/0	1/1/1	대기, 버스 해제
1/0	0/1/0	FPGA가 쓰기 데이터 구동
0/1	0/0/1	SRAM 출력 읽기
1/1	1/1/1	동시 요청 무시

예상 결과와 검증 계획

주소 i에 15−i를 쓰고 다시 읽는다. 예를 들어 주소 0은 0xF, 주소 5는 0xA, 주소 15는 0x0이 LED 하위 4비트에 표시되어야 한다. LED 상위 4비트는 0이며 읽기 이외 상태에서는 전체 LED 출력이 0이 된다.

SRAM 테스트벤치

주소 0~15에 서로 다른 값을 저장하고 전부 읽어 비교한다. 대기 상태에서 ram_data가 Z인지, 읽기 중 FPGA가 버스를 해제하는지, 동시 요청에서 CS/OE/WE가 모두 HIGH인지 확인한다. 보드에서는 쓰기 후 입력 데이터를 바꾸어도 저장한 값이 읽히는지 확인한다.

7. FPGA DAC 출력

설계 목표와 회로 구성

ramp8에서 8비트 값 0~255를 순환 생성하고 dac_parallel8이 데이터와 쓰기 신호를 외부 DAC에 전달한다. 최상위 dac는 채널 선택과 LED 출력을 연결한다. STM32 내부 DAC가 아니라 FPGA로 외부 DAC를 제어한다.

데이터와 쓰기 펄스

```
assign update =
    (count == UPDATE_CYCLES-1);

always @(posedge clk_50mhz)
  if (rst_p) begin
    we_n <= 1;
    a_b <= 0;
    data <= 0;
  end else begin
    we_n <= 1;
    if (update) begin
      data <= value;
      a_b <= channel;
      we_n <= 0;
    end
  end
```

갱신 이벤트에서 데이터와 채널을 등록하고 we_n을 한 클록 동안 LOW로 만든다. 그 다음 상승 에지에서 we_n이 HIGH로 복귀하며 데이터는 유지된다. 코드상 데이터 변경과 LOW 시작은 같은 클록 에지에 일어나므로 실제 DAC의 래치 에지와 설정 시간 조건을 확인해야 한다.

예상 시간과 출력

항목	기본 설정의 계산값
기준 클록	50 MHz, 주기 20 ns
갱신 간격	500,000클록 = 10 ms
갱신 속도	100회/s
한 순환	256단계 = 2.56 s
쓰기 LOW 폭	1클록 = 20 ns

LED에는 DAC에 전달한 8비트 코드가 나타나며 255 다음에는 0으로 돌아간다. DAC 출력은 단계적으로 증가한 뒤 초기값으로 돌아오는 형태를 예상한다. 단극성 이상적 8비트 DAC를 가정하면 출력은 VREF×D/256에 비례하지만 실제 출력 전압은 장치의 기준 전압과 출력 회로를 확인한 뒤 비교한다.

DAC 테스트벤치

UPDATE_CYCLES=4로 가속하여 260회 쓰기를 검사하고 255→0 전환, 채널 선택, 데이터 유지, 리셋을 확인한다. TB의 클록 주기는 10 ns이며 LOW 시작 후 1 ns 뒤 시간을 기록하므로 종료 에지와의 차이는 9 ns로 비교한다. 실제 펄스 폭은 1클록인 10 ns이다.

보드에서는 갱신 간격 10 ms와 출력 순환 약 2.56 s를 기준으로 디지털 버스와 아날로그 파형을 확인한다. LED 값만으로 출력 전압이나 DAC 선형성을 확인했다고 판단하지 않는다.

8. 소스 구성과 Github주소

소스와 실행 구성

파일	역할
ece2_474_routine.c	과제 5·6·7 수신·전달
ece2_474_stdio.c	UART2 표준 입출력
main.c / MSP	UART 및 핀 초기화
CMakePresets.json	Task5·Task6·Task7 빌드
FPGA_additional.md	서보·SRAM·DAC와 TB

- GITHUB주소: https://github.com/thechef030/STM32_2
- 브랜치: stm32-2
- 현재 소스 기준 커밋: 4dbaf0af8425f039666ff7f13a39ec413b6c8ea6
- 태그: v1.1.0

9. 참고문헌

[1] 전전설II LAB6 교안

[2] STMicroelectronics, STM32CubeG4 MCU Firmware Package 및 STM32G4 HAL UART 드라이버. UART 초기화와 폴링 송수신 API를 참조.